

scrartcl [T1]fontenc [romanian]babel [dvipsnames]xcolor microtype comment listings ams-math,amsthm [libertine]newtx bm tikz hyperref [nameinlink]cleveref inconsolata

Autor problemă: PROF. EMIL CERCHEZ

Rezumat

Se consideră un șir de n cifre zecimale s . Trebuie să alegem exact p cifre cărora le vom pune semnul $+$, exact q cifre cărora le vom pune semnul $-$, iar restul de $r = n - p - q$ cifre vor rămâne fără semn. Dintre toate subsecvențele de lungime r , trebuie aleasă cea lexicografic maximă. Apoi trebuie calculată valoarea expresiei rezultate.

Observații cheie

- Lungimea subsecvenței care rămâne fără semn este $r = n - p - q$.
- Dintre toate subsecvențele de lungime r din s , trebuie aleasă cea lexicografic maximă.
- Cifrele rămase (cele care nu fac parte din subsecvența maximă) pot fi semnate arbitrar, dar trebuie exact p semne $+$ și q semne $-$.
- Pentru a maximiza valoarea expresiei, cifrele cu semnul $+$ trebuie să fie cât mai mari, iar cifrele cu semnul $-$ trebuie să fie cât mai mici. Acest lucru se realizează sortând cifrele rămase $-$ primele q (cele mai mici) primesc $-$, restul primesc $+$.
- Rezultatul poate depăși capacitatea tipurilor standard, deci trebuie implementată aritmetica cu numere mari.

Soluție

Pasul 1: Găsirea subsecvenței lexicografic maxime

Parcurgem toate pozițiile posibile de start pentru o subsecvență de lungime r : de la 0 la $n - r$. Comparăm fiecare subsecvență cu cea curent maximă folosind comparație lexicografică standard.

```
imax = 0;  
if (p > 0)  
    for (i = 1; i < n - r + 1; i++)  
        if (compars(i, imax) > 0) imax = i;
```

Funcția `compars` compară cele două subsecvențe caracter cu caracter.

Pasul 2: Extragerea cifrelor rămase

După ce am găsit poziția $i_{\max}$ a subsecvenței maxime, extragem:

- dacă $i_{\max} = 0$: toate cifrele de după subsecvență
- altfel: cifrele de dinaintea de $i_{\max}$ (exceptând prima cifră care devine parte din calculul sumei inițiale) și cifrele de după subsecvență

Aceste cifre sunt plasate în vectorul a și sortate în ordine crescătoare.

Pasul 3: Determinarea sumei parțiale

Cifrele din a sunt împărțite în două grupuri:

- primele q cifre (cele mai mici) primesc semnul $-$
- ultimele p cifre primesc semnul $+$

Suma algebrică a acestor cifre (fără subsecvența maximă) este calculată în variabila $suma$.

Pasul 4: Operația finală

Fie M subsecvența lexicografic maximă (lungime r) și S suma calculată anterior. Rezultatul final este $M \pm S$, unde semnul depinde de compararea lui M și S :

- dacă $S \geq 0$ și $|S|$ se adaugă la M
- dacă $S < 0$: comparăm M cu $|S|$
 - dacă $M \geq |S|$: rezultat = $M - |S|$
 - altfel: rezultat = $|S| - M$, cu semn negativ

Implementarea operațiilor cu numere mari

Tipul `NrMare` reprezintă un număr mare ca un vector de cifre în ordine inversă (cifra unităților pe poziția 0).

Adunarea

```
void adun(NrMare x, int lgx, NrMare y, int lgy, NrMare z, int& lgz)
{
    if (lgx < lgy) {
        lgz = lgy;
        for (i = lgx; i < lgy; i++) x[i] = 0;
    } else {
        lgz = lgx;
```

```

        for (i = lgy; i < lgx; i++) y[i] = 0;
    }
    for (t = i = 0; i < lgz; i++) {
        val = x[i] + y[i] + t;
        z[i] = val % 10;
        t = val / 10;
    }
    if (t) z[lgz++] = t;
}

```

Se egalizează lungimile prin completarea cu zerouri, apoi se adună cifră cu cifră propagând transportul.

Scăderea

Funcția `dif` implementează $z = x - y$, presupunând $x \geq y$. Se gestionează împrutul când diferența devine negativă.

Compararea

Funcția `comparnr` compară două numere mari: mai întâi după numărul de cifre, apoi cifră cu cifră dacă au aceeași lungime.

Complexitate

- Căutarea subsecvenței maxime: $O(n \cdot r)$ în cazul cel mai defavorabil, dar $r \leq n$, rezultând $O(n^2)$ — acceptabil pentru $n \leq 1000$.
- Sortarea cifrelor rămase: $O(n \log n)$.
- Operațiile cu numere mari: $O(r)$, unde $r = n - p - q$.

Corectitudine

Lema 1: Subsecvența lexicografică maximă de lungime r este plasată obligatoriu în poziția $i_{\max}$ găsită de algoritm.

Demonstrație: Algoritmul verifică toate cele $n - r + 1$ poziții posibile și compară lexicografic fiecare subsecvență cu cea curent maximă. La final, $i_{\max}$ conține indicele subsecvenței maxime. $\square$

Lema 2: După sortarea crescătoare a cifrelor rămase, primele q cifre sunt cele mai mici posibile care pot primi semnul $-$.

Demonstrație: Sortarea asigură ordinea crescătoare. Pentru a minimiza scăderea totală, cifrele cu semnul $-$ trebuie să fie cât mai mici. $\square$

Lema 3: Atribuirea semnelor $+/-$ astfel încât primele q cifre sortate primesc $-$ și restul primesc $+$ maximizează valoarea expresiei.

Demonstrație: Expresia are forma $M + \sum_+ d_i - \sum_- d_j$. Pentru M fix, trebuie să maximizăm diferența $\sum_+ d_i - \sum_- d_j$. Aceasta se realizează luând cele mai mici q cifre pentru scădere și cele mai mari p cifre pentru adunare. $\square$

Teorema: Algoritmul produce valoarea corectă a expresiei finale.

Demonstrație:

1. Conform Lemei 1, identificăm corect subsecvența lexicografică maximă M .
2. Conform Lemelor 2 și 3, calculăm corect suma S a cifrelor semnate optimal.
3. Calculăm $M \pm S$ folosind aritmetica cu numere mari, acoperind toate cazurile de semn.

Prin urmare, rezultatul afișat este exact valoarea expresiei cerute. $\square$